

實作完成。新建 `app/Providers/image_gen_provider/`，完全遵循 `llm_provider` 的 Singleton 架構模式。

檔案結構

```
app/Providers/image_gen_provider/ |—— init.py # 模組匯出 |—— client.py #  
ImageGenClient - pipeline wrapper |—— manager.py # ImageGenManager - Singleton +  
threading.Lock 使用方式
```

```
from app.Providers.image_gen_provider import get_image_gen_manager
```

方式 1: 透過 Manager (推薦)

```
manager = get_image_gen_manager() image = manager.generate("An astronaut riding a  
green horse") manager.release() # 用完釋放 VRAM
```

方式 2: FastAPI DI

```
from fastapi import Depends
```

```
@router.post("/generate") async def generate_image( manager: ImageGenManager =  
Depends(get_image_gen_manager) ): image = manager.generate(prompt="A cat in space",  
num_inference_steps=30) ...
```

方式 3: 多次呼叫 — 模型只載入一次

```
m1 = get_image_gen_manager() m2 = get_image_gen_manager() assert m1 is m2 # True —  
同一 Singleton 與 LLMManager 的架構對照 模式 LLMManager ImageGenManager Singleton  
new + _initialized 相同 Lock asyncio.Lock threading.Lock (torch 同步) Lazy load get_client()  
get_client() 釋放資源 shutdown() shutdown() / release() DI factory get_llm_manager()  
get_image_gen_manager() COMPLIANCE CONFIRMED: 遵循現有架構模式，新增模組因  
domain 差異 (圖像生成 vs 文本處理) 無法擴展現有 Provider。
```